

สรุปความรู้: SDLCและSRS (Homework#3)

➤ ส่วนที่ 1: SDLC (Software Development Life Cycle)

1. ความหมายของ SDLC

SDLC (Software Development Life Cycle) หรือวัฏจักรการพัฒนาซอฟต์แวร์ คือ กระบวนการที่เป็นระบบและมีขั้นตอนชัดเจนในการสร้าง ปรับปรุง และบำรุงรักษาซอฟต์แวร์ โดยเริ่มตั้งแต่การรับความต้องการจากผู้ใช้งานไปจนถึงการส่งมอบและดูแลระบบหลังการใช้งานจริง เป้าหมายหลักของ SDLC คือการช่วยให้ทีมพัฒนาสามารถส่งมอบซอฟต์แวร์ที่มีคุณภาพสูง ตรงตามความต้องการของลูกค้า ภายใต้กรอบเวลาและงบประมาณที่กำหนดไว้ (Sommerville, 2011)

2. ขั้นตอนของ SDLC

ขั้นตอนของ SDLC ประกอบด้วย 6 เฟสหลัก เริ่มต้นจาก **Planning** (การวางแผนและศึกษาความเป็นไปได้) และ **Analysis** (การวิเคราะห์ความต้องการของระบบ) เพื่อทำความเข้าใจปัญหา จากนั้นจะเข้าสู่ขั้นตอน **Design** (การออกแบบสถาปัตยกรรมและหน้าตาของระบบ) เพื่อวางโครงสร้างก่อนจะนำไปสู่ขั้นตอน **Implementation** (การเขียนโค้ดและสร้างระบบจริง) โดยโปรแกรมเมอร์หลังจากได้ตัวระบบแล้ว จะต้องผ่านขั้นตอน **Testing** (การทดสอบระบบ) เพื่อค้นหาและแก้ไขข้อผิดพลาด (Bugs) ให้เรียบร้อย ก่อนที่จะทำการติดตั้งเพื่อให้ผู้ใช้ใช้งานจริง และปิดท้ายด้วยขั้นตอน **Maintenance** (การบำรุงรักษา) ซึ่งเป็นการดูแลและปรับปรุงระบบให้พร้อมใช้งานและทันสมัยอยู่เสมอเมื่อสภาพแวดล้อมทางธุรกิจเปลี่ยนไป (Pressman, 2014)

3. Waterfall Model

Waterfall Model (โมเดลน้ำตก) เป็นแนวคิดการพัฒนาซอฟต์แวร์แบบดั้งเดิมที่มีลักษณะการทำงานเรียงลำดับเป็นขั้นบันไดลงมาทีละเฟสอย่างชัดเจน โดยมีเงื่อนไขสำคัญคือ ขั้นตอนก่อนหน้าจะต้องเสร็จสมบูรณ์และได้รับอนุมัติ 100% ก่อน จึงจะสามารถเริ่มขั้นตอนถัดไปได้ (เช่น **ต้องวิเคราะห์ความต้องการให้เสร็จสิ้นทั้งหมดก่อน จึงจะเริ่มออกแบบโครงสร้างฐานข้อมูลได้**)

โมเดลนี้มีข้อดีคือ การบริหารจัดการง่าย มีเอกสารควบคุมในทุกขั้นตอน และคาดการณ์งบประมาณได้ชัดเจน แต่มีข้อจำกัดอย่างมากในเรื่องความยืดหยุ่น หากผู้ใช้งานต้องการแก้ไขความต้องการ

ในภายหลัง จะทำได้ยากและมีค่าใช้จ่ายสูงมาก จึงเหมาะสำหรับโครงการที่ข้อกำหนดนิ่งและไม่มี การเปลี่ยนแปลง (Sommerville, 2011)

4. Agile Model

Agile Model เป็นแนวคิดการพัฒนาซอฟต์แวร์ที่เน้นความยืดหยุ่น การปรับตัว และการส่งมอบงาน ที่รวดเร็ว โดยจะปฏิเสธการวางแผนระยะยาวที่ตายตัว แต่จะแบ่งการทำงานออกเป็นรอบสั้นๆ เรียกว่า Iteration หรือ Sprint (**มักใช้เวลา 1–4 สัปดาห์**) ในแต่ละรอบทีมจะเลือกฟีเจอร์ย่อยๆ มา พัฒนา ทดสอบ และส่งมอบชิ้นงานที่ใช้งานได้จริงให้ลูกค้าประเมินทันที

แนวคิดนี้ให้ความสำคัญกับการสื่อสารภายในทีมและการทำงานร่วมกับลูกค้าอย่างใกล้ชิด ทำให้ สามารถรองรับการเปลี่ยนแปลงของความต้องการ (**Requirements**) ได้ตลอดเวลา เหมาะอย่างยิ่ง กับโครงการยุคใหม่ที่มีความผันผวนสูงและต้องการส่งผลิตภัณฑ์ออกสู่ตลาดอย่างรวดเร็ว (Schwaber & Beedle, 2002)

5. ความสำคัญของ SDLC ในการพัฒนาซอฟต์แวร์

SDLC มีความสำคัญอย่างยิ่งเนื่องจากเปรียบเสมือน "**แผนที่นำทาง (Roadmap)**" ที่ช่วยให้ทีม พัฒนาเข้าใจเป้าหมายและขั้นตอนการทำงานตรงกัน ลดความสับสนและการทำงานซ้ำซ้อน นอกจากนี้ยังช่วยให้ผู้บริหารโครงการสามารถควบคุมงบประมาณ ระยะเวลา และทรัพยากรได้ อย่างมีประสิทธิภาพ ตลอดจนลดความเสี่ยงที่โครงการจะล้มเหลวหรือได้ซอฟต์แวร์ที่ไม่ตรงตาม ความต้องการของธุรกิจ (Pressman, 2014)

➤ **ส่วนที่ 2: SRS(Software Requirements Specification)**

6. ความหมายของ SRS

SRS (Software Requirements Specification) หรือเอกสารข้อกำหนดซอฟต์แวร์ คือ เอกสารที่เป็นทางการซึ่งรวบรวมและระบุความต้องการทั้งหมดของระบบซอฟต์แวร์ที่จะพัฒนาขึ้น ทำหน้าที่ เป็นสัญญาร่วมกันระหว่างทีมพัฒนาระบบและผู้ว่าจ้าง (**หรือผู้ใช้งาน**) เพื่อให้ทั้งสองฝ่ายมีความ เข้าใจที่ตรงกันอย่างสมบูรณ์เกี่ยวกับสิ่งที่ซอฟต์แวร์จะต้องทำ และข้อจำกัดต่างๆ ที่ต้องปฏิบัติตาม ก่อนที่จะเริ่มกระบวนการออกแบบและเขียนโค้ด (IEEE Computer Society, 1998)

7. โครงสร้างของเอกสาร SRS

โครงสร้างมาตรฐานของเอกสาร SRS (มักอ้างอิงตามมาตรฐาน IEEE 830) แบ่งออกเป็น 3 ส่วนหลัก ได้แก่:

- **Introduction (บทนำ):** ระบุวัตถุประสงค์ ขอบเขตของระบบ และคำนิยามต่างๆ
- **Overall Description (ภาพรวมของระบบ):** อธิบายความสัมพันธ์ของระบบกับปัจจัยภายนอก ฟังก์ชันหลักๆ แบบย่อ และข้อจำกัดทั่วไปของโครงการ
- **Specific Requirements (ข้อกำหนดเฉพาะ):** ส่วนที่ลงลึกในรายละเอียดของระบบทั้งหมด เช่น ฟังก์ชันการทำงานโดยละเอียด (Functional) และเกณฑ์ประสิทธิภาพ(Nonfunctional) เพื่อให้โปรแกรมเมอร์สามารถนำไปใช้พัฒนาต่อได้ทันที (IEEE Computer Society, 1998)

8. Functional Requirements

Functional Requirements คือ ข้อกำหนดที่ระบุว่า **"ระบบต้องทำอะไรได้บ้าง (What the system should do)"** ซึ่งเป็นฟังก์ชันการทำงาน บริการ หรือสิ่งที่เป็นผลลัพธ์โดยตรงที่ระบบต้องมอบให้กับผู้ใช้งาน ตัวอย่างเช่น "ระบบต้องมีฟังก์ชันให้ผู้ใช้สมัครสมาชิกผ่าน Email ได้", "ระบบต้องคำนวณราคาสินค้ารวมภาษีมูลค่าเพิ่ม 7% ได้อย่างถูกต้อง" หรือ "ระบบต้องส่งข้อความแจ้งเตือนเมื่อการชำระเงินเสร็จสิ้น" เป็นต้น (Sommerville, 2011)

9. Non-functional Requirements

Non-functional Requirements คือ ข้อกำหนดที่ระบุว่า **"ระบบต้องทำงานอย่างไร หรือมีคุณสมบัติอย่างไร (How the system should perform)"** ซึ่งไม่ใช่ฟังก์ชันการทำงานโดยตรง แต่เป็นเกณฑ์เชิงคุณภาพที่ช่วยให้ระบบทำงานได้อย่างมีประสิทธิภาพ มั่นคง และปลอดภัย ตัวอย่างเช่น "ระบบต้องสามารถรองรับผู้ใช้งานพร้อมกันได้ 10,000 คน", "หน้าเว็บต้องใช้เวลาโหลดไม่เกิน 2 วินาที" หรือ "ข้อมูลรหัสผ่านของผู้ใช้ทั้งหมดต้องถูกเข้ารหัสด้วย SHA-256" (Pressman, 2014)

10. ความสำคัญของ SRS ในโครงการซอฟต์แวร์

เอกสาร SRS เป็นหัวใจสำคัญของโครงการซอฟต์แวร์ เพราะช่วยป้องกันปัญหา **"ความเข้าใจคลาดเคลื่อน"** ระหว่างลูกค้าและทีมพัฒนา ซึ่งเป็นสาเหตุหลักที่ทำให้โครงการล้มเหลว นอกจากนี้ SRS ยังใช้เป็นเกณฑ์อ้างอิงในการตรวจสอบและทดสอบระบบ (Software Testing) ว่าระบบที่พัฒนาเสร็จแล้วทำงานได้ถูกต้องครบถ้วนหรือไม่ รวมถึงช่วยให้ฝ่ายบริหารสามารถประเมินราคาและระยะเวลาของโครงการได้อย่างแม่นยำ (Sommerville, 2011)

➤ เอกสารอ้างอิง (References)

- IEEE Computer Society. (1998). IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications. IEEE.
 - Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.
 - Schwaber, K., & Beedle, M. (2002). Agile Software Development with Scrum. Prentice Hall.
 - Sommerville, I. (2011). Software Engineering (9th ed.). Addison-Wesley
-